

OPERATING SYSTEMS FOR EMBEDDED SYSTEMS

An embedded system is a specialized programming device designed to perform specific tasks or intervals within a larger system. Unlike general-purpose computers, embedded systems are specialized for specific applications and typically operate within a defined hardware and software environment. These systems integrate the performance of hardware, such as microcontrollers or microprocessors, with software to control, monitor, or enable physical senses. An embedded system is typically part of a vehicle, device, or industrial machine where it functions autonomously, often in real time (*Digi International, 2025*). These embedded systems can operate with little or no human intervention (*GeeksforGeeks, 2025*). Embedded systems are systems that generally have a limited interface or no interface, are difficult to optimize criteria such as power consumption, processor power, code size, cost and life, and combine features such as real-time operation, reliability, expected lifespan and the development of necessary software in a single structure. (*DergiPark, 2021*) Embedded systems are called "embedded" because they do not operate in isolation; They are "embedded" in larger devices and electronic systems to control, monitor, or automate tasks in industries such as manufacturing, automotive, consumer electronics, and healthcare (*C&T Solution, 2025*). This computing system integrates a combination of processor, memory, I/O, and software to accomplish a specific purpose. It can operate independently or as a component of a larger system (*Teguar, 2025*). The three main components of embedded systems are hardware, software/firmware, and RTOS. Hardware consists of compact, low-power units such as microcontrollers or microprocessors; software and firmware contain the specialized code that runs the system; and RTOS provides real-time processing by responding quickly to external events (*C&T Solution, 2025*).

A real-time operating system (RTOS) is an operating system with two key properties: predictability and determinism. In an RTOS, repetitive tasks are performed within a time limit; However, this is not always the case in a general-purpose operating system. In this case, predictability and determinism go hand in hand. An embedded system is a computer built into a larger machine (for example, a microcontroller in a robotic arm). The features that distinguish an RTOS from others are often essential for success. For example, a robotic arm in a factory needs to be predictable and reliable, and able to stop instantly when workers enter the work area. Variability can lead to resource waste, quality control issues, or injuries (*Wind River, 2025*).

These systems improve device performance by ensuring efficient and safe operation of hardware and provide automation and intelligent functions; they also make technology more accessible thanks to their small size and low cost (*Lim, 2014*). Embedded systems are designed to perform specific tasks and are commonly used in everyday applications such as calculators, washing machines, and drones. These systems increase performance and reliability by enabling the integrated operation of hardware and software within each device (*DeepSea Developments, 2022*).

This report provides performance analysis, history and design features used in embedded systems, reveals existing security vulnerabilities, and offers recommendations for current designs. The report will first cover the background and history of integrated systems; Then, different operating system designs and architectures will be examined through literature review; The current

potential security will be explained and finally the system parts, quantities and development suggestions to increase its usability will be presented.

The development of microprocessors coincided with the emergence of embedded operating systems in the late 1960s and early 1970s, when simple firmware was the primary means of controlling embedded devices (*Stallings, 2017*). Early embedded systems didn't initially need a full operating system because they were made to carry out a single specific task with incredibly little memory and processing power. However, the need for predictable and structured execution models increased dramatically as these devices started to be used in time-sensitive environments, such as telecommunications, avionics, and military equipment (*Tanenbaum & Bos, 2015*).

Early commercial real-time kernels were developed as a result of this change. The 1981 release of VRTX32, one of the first commercially successful real-time operating system (RTOS) kernels with deterministic task scheduling, is one of the most prominent examples (*Labrosse, 2009*). These kernels signaled the shift from straightforward firmware-driven systems to more intricate task-oriented embedded architectures.

By the middle of the 1980s, there was a significant need for real-time operating systems that could handle several tasks under tight time constraints due to the increasing complexity of embedded applications. After its release in 1987, VxWorks swiftly rose to prominence as one of the most popular commercial RTOS programs. It was appropriate for safety-critical applications due to its deterministic scheduler, low interrupt latency, and modular architecture (*Wind River Systems, 2019*). In the same time frame, QNX became well-known in automotive, industrial, and medical systems as a microkernel-based RTOS with remarkable fault isolation and reliability (*QNX Software Systems, 2018*). Predictability is a key feature of RTOS platforms. RTOS schedulers, in contrast to general-purpose systems, are required to ensure that tasks fulfill stringent deadlines, frequently in milliseconds. Because of this, RTOS solutions are crucial in settings like implantable medical devices, avionics systems, and electronic control units (ECUs), where even slight timing errors can cause harm or safety risks (*Burns & Wellings, 2016*).

Microprocessor performance advancements in the 1990s and 2000s made it possible for embedded systems to run more feature-rich operating systems. Consequently, proprietary RTOS solutions were no longer the only practical choice. Due to its flexibility, broad hardware support, and affordability, Embedded Linux became widely used as a result of the growth of open-source software (*Yaghmour, 2003*). Embedded operating systems based on Linux quickly gained popularity in networking devices, consumer electronics, and industrial automation.

Google developed Android, which is based on the Linux kernel and supports more user-centric embedded platforms, in response to the growth of mobile and Internet of Things devices. This model was later expanded to IoT-specific devices by Android Things (*Google, 2018*). In the meantime, TinyOS, a lightweight embedded operating system, was created especially for wireless embedded devices and low-power sensor networks, allowing effective operation on incredibly limited hardware (*Levis et al., 2004*). This variety illustrates the expanding range of embedded systems and their quickly changing software needs.

Analyzing the development of embedded platforms requires an understanding of the differences between RTOS and general-purpose operating systems (GPOS). While GPOS solutions like Windows or conventional Linux distributions concentrate on throughput, multitasking, and user experience, an RTOS emphasizes determinism, guaranteeing that tasks finish within predetermined time constraints (*Stallings, 2017*).

While time-critical and safety-critical systems continue to be dominated by RTOS platforms, embedded multimedia systems, infotainment platforms, robotics, and smart IoT devices are increasingly using GPOS variants, especially Linux (*Tanenbaum & Bos, 2015*). Systems like VxWorks, QNX, and FreeRTOS are still favored in deterministic settings today, while Linux, Android, and related platforms are used for applications that need more sophisticated interfaces and processing power.

Embedded real-time operating systems (RTOS) are specialized software used to determine behavior, reduce latency, and increase reliability. In this context, our literature will cover the core architecture that defines RTOS architecture, scheduling algorithms, memory management, task communication and synchronization, and comparisons of different embedded systems. We will utilize articles of high academic value and widely studied worldwide to address these topics.

Our literature review will examine RTOS architecture in five different sections. The first of these sections is Kernel Architecture. Kernel architecture, used in real-time operating systems to determine system stability and deterministic behavior, is crucial for RTOS. Kernel architecture is further divided into specific architectures: Monolithic Kernels and Microkernels. Monolithic kernels provide performance advantages for RTOSs, while microkernels are primarily used to provide services to users, identify vulnerabilities, and reduce their fault tolerance. QNX, which conducts research on microkernel technologies, has stated that these systems can be effectively used in industrial applications and security issues today, based on their structure (*Hildebrand, 1992*).

Another topic addressed in the literature regarding RTOS architecture is Scheduling. Important terms for scheduling – rate-monotonic and earliest-deadline-first algorithms- were defined in the classic work of *Liu and Layland (1973)*, and this definition formed the theoretical basis of real-time systems. For these systems *Joseph and Pandya (1986)* generated an analysis that RTOS scheduling analysis has also been addressed by response times of various tasks under the worst-case scenario using analytical methods. This study made a significant contribution to RTOS scheduling analysis.

One of the important topics discussed in the literature is Memory Management. Memory management is a key factor distinguishing embedded systems from standard operating systems and, therefore, has been addressed effectively in literature. Its limited capacity and lack of deterministic behavior cause difference between systems. In RTOS architecture, memory, due to its unpredictable nature, can cause some omissions and problems in the field due to jitter and time. Therefore, static memory allocation or predictable memory structures are preferred today. Memory analysis has gained importance for researchers, *Wehmeyer and Marwedel (2007)* clearly emphasized the impact of predictability in memory hierarchies on RTOS architecture in their analysis of memory management in RTOS architecture.

Another topic we will address in our literature is Task Communication and Synchronization. In RTOS architecture, mechanisms such as message queues, semaphores, and mutexes play a critical role in inter-task data transfer and resource sharing. Furthermore, the mutex structure is effectively used to mitigate the priority inversion problem, which is common in embedded systems (*Sha et al., 1990*). The latencies of these mechanisms directly influence RTOS selection.

Finally, our literature has examined various embedded systems. This section of our literature discusses the features of platforms such as FreeRTOS, VxWorks, QNX, Linux-based RT solutions, and TinyOS. FreeRTOS offers low-cost minimal architecture for embedded devices. Furthermore, VxWorks exhibits deterministic behavior in industrial applications. TinyOS, with its specialized design for sensor networks, provides lightweight architecture. The latency, IPC performance, and reliability values of these systems, measured on real hardware, are crucial for RTOS selection. These values indicate that different RTOS selections are necessary for specific applications (*Yoon et al., 2013*).

To sum up, our literature has addressed the Kernel Architectures, Scheduling Algorithms, Memory Management, Task Communication and Synchronization, all of which are important in developing RTOS architectures, as well as comparisons of different embedded systems. In addition to fundamental architectural issues, our literature has also addressed IPC performance, reliability, and the reasons for selecting different RTOS architectures.

Security is the ability of an entity to protect resources for which it bears responsibility. In an embedded system, this responsibility applies to resources within the system or those of the larger overall system to which it is connected (*Kleidermacher & Kleidermacher, 2012*).

Embedded systems are the driving force in many application domains that build smart or intelligent systems, including automotive electronics, avionics, consumer electronics, railways, telecommunications, and healthcare (*Marwedel, 2010*). As attacks on cyber systems can cause physical damage, security is a fundamental requirement (*Langner, 2011*).

However, compared to traditional IT platforms, embedded system security is often weaker due to limited resources, outdated design practices, and the difficulty of continuous patching. Wide deployment scale, physical exposure, cost constraints, and remote accessibility make embedded operating systems particularly vulnerable (*Serpanos & Voyiatzis, 2013*). Below are the major vulnerability categories identified across research and real-world cases.

1. Programming Errors:

Many of the vulnerabilities stem from programming errors, which may lead to control flow attacks (Papp, Ma, & Buttyán, 2015). These errors include **Buffer Overflow** where excess data written to a buffer overwrites adjacent memory, potentially allowing arbitrary code execution (Fiveable). Another major issue is **Memory Management Problems**, like Using pointers referring to memory locations that have been freed.

Possible Consequences:

- Remote Code Execution (RCE) through exploitation of overflow bugs.
- Denial-of-Service (DoS) caused by forced system crashes.
- Unauthorized firmware modification or persistent compromise

2. Web Based Vulnerability:

Many embedded devices have a web-based management interface for configuration and updates. However, the web server applications running on those devices are typically rarely updated. Hence, those devices are exposed to web-based attacks that exploit unpatched vulnerabilities in the web-based interface of the device (*Papp, Ma, & Buttyán, 2015*).

Possible Consequences:

- Credential theft through HTTP interception or session hijacking.
- Man-in-the-Middle (MitM) attacks due to missing TLS enforcement.

3. Weak Access Control or Authentication:

Many embedded devices have a web-based management interface for configuration and updates. However, web server applications are often rarely updated. Hence, those devices are exposed to attacks that exploit unpatched vulnerabilities in the interface of the device (*Papp, Ma, & Buttyán, 2015*).

Possible Consequences:

- Unauthorized users obtain administrative control.
- In safety-critical environments (e.g., medical, automotive), may escalate to physical harm.

4. Improper Use of Cryptography:

Cryptographic mechanisms used for authentication or confidentiality of some sensitive information often result in fatal security failures due to inappropriate implementation. Examples include using weak random number generators for cryptographic keys, or vulnerabilities in the cryptographic protocols themselves (*Papp, Ma, & Buttyán, 2015*).

Possible Consequences:

- Exposure of sensitive data such as credentials or encryption keys.
- Successful replay, spoofing, or downgrading attacks on communication channels.

5. Outdated or Non-Patchable Firmware:

Embedded OSs are used in the field for a long time - more than a decade- without updates. Many organizations struggle to keep their systems fully updated. When patches are not applied, known vulnerabilities remain exploitable for years. (*Splashtop, 2025*).

Possible Consequences:

- Exploitation of well-known CVEs. (*Splashtop, 2025*).
- Malicious firmware installation if verification is lacking.
- Industrial systems may face downtime, production failure, or physical damage.
- Potential spread of compromise to critical infrastructure.

In conclusion, embedded OSs remain highly exposed to security threats. Limited resources, long device lifecycles, and weak update mechanisms leave many systems vulnerable to attacks. Ensuring the security of embedded OSs is therefore essential for protecting not only data, but also the physical systems they control.

The security landscape of IoT devices and embedded operating systems has witnessed rapid evolution. These have evolved from simple gadgets to key components of critical infrastructures, medical devices, and smart homes. Along with such rising stakes, the potential for attacks has grown manifold. This new world requires a very different mindset from the “**patch-it-later**” approach in which security is treated as an afterthought. No longer will long-term resilience be achieved without taking a **holistic lifecycle approach**, integrating security from initial design through eventual removal. This section provides guidance on design and operational practices and discusses emerging trends that will shape the future of embedded security.

Building a secure device is very similar to building a house: weak foundation means the whole structure is at risk. Thus, it must be Secure-by-Design. It means that security needs to be considered even before any line of code has been written. One such key practice would be threat modeling frameworks such as Microsoft's STRIDE – *Spoofing*, *Tampering*,

Repudiation, *Information Disclosure*, *Denial of Service*, and *Elevation of Privilege* (*Shostack, 2014*). The systematic analysis of the system design through STRIDE means architects are able to find issues in a logical or ineffective way, with potential attack paths that might not be seen by automated tools later in time. Besides the choice of language, coding standards are strict. While C and C++ are considered the industrial standard languages within the embedded systems environment, they include quite a number of memory-related attacks. Therefore, their guidelines such as MISRA C or CERT C (*Seacord, 2013*) should be followed. In addition, incorporation of SAST tools into the CI/CD pipeline allows for automatic detection of weaknesses at the build process in order for problems to be fixed ahead of time of release.

From a technical viewpoint, trust should start with hardware. A robust mechanism for Secure Boot becomes mandatory to make sure the device will only boot software that is digitally signed and trusted by the manufacturer. Cryptographic keys and biometric data must be protected through hardware-based isolation in the form of TEEs such as ARM TrustZone. Protection must also extend to data at rest and data in transit through strong encryption like AES-256 and protocols including TLS 1.3. Because an IoT device could live for ten years or more, there must be a secure Over-the-Air (OTA) update mechanism with signed updates and anti-rollback functionality. Network structures need to shift towards a Zero Trust model with microsegmentation and strict Role-Based Access Control (RBAC) to impede lateral movement in cases of compromise.

Security is never a point-in-time activity; it's a continuous process. Operational resilience demands rapid responses to newly identified threats. Organizations should establish a rapid Patch Management process, informed by the maintenance of an SBOM (Software Bill of Materials), which will allow teams to quickly identify which devices are impacted on when vulnerabilities are discovered in third-party components. Monitoring needs to be active, with devices creating tamper-resistant logs transferred to a central SIEM system for near real-time anomaly detection and incident response.

Looking ahead, embedded software will be harder to verify. Industry is working towards Formal Verification, using mathematical techniques to confirm correctness and eliminate many bugs. AI will also play on both sides: attackers will use AI to automate exploitation, while defenders will rely on AI-powered IDS to recognize zero-day attacks through behavioral analysis. Finally, the regulatory landscape is changing devices will have to upgrade to Post-Quantum Cryptography as quantum computing advances; and regulations such as the EU Cyber Resilience Act will demand security by default and long-term responsibility of manufacturers for product security.

In conclusion, embedded systems, which are specialized computing devices integral to fields ranging from automotive to healthcare, rely on predictable and deterministic Real-Time Operating Systems (RTOS) to manage tasks under strict time constraints. The analysis of kernel architecture, scheduling, and memory management highlights the need for specialized design methodologies to ensure reliability and performance in these critical environments.

However, the proliferation of these devices makes them highly exposed to security threats. Compared to traditional IT platforms, embedded system security is often weaker due to limited resources, outdated design practices, and the difficulty of continuous patching. Major vulnerabilities stem from programming errors, weak authentication, and the use of outdated, non-patchable firmware, which can lead to remote code execution and even physical harm.

Addressing these persistent threats requires a foundational shift toward a Secure-by-Design philosophy, integrating security from the initial concept phase. Future security must focus on robust hardware-based isolation (TEEs), secure Over-the-Air (OTA) updates, and continuous monitoring, alongside the adoption of emerging concepts like Formal Verification and AI-powered defenses. Ensuring the security of embedded OSs is therefore essential for protecting not only data but also the physical systems they control.

REFERENCES

1. Digi International. (2025). Embedded system.
2. GeeksforGeeks. (2025). Introduction of embedded systems (Set 1).
3. DergiPark. (2021). Embedded systems.
4. C&T Solution. (2025). What is an embedded system?
5. Tegar. (2025). What is an embedded system?
6. Wind River. (2025). Real-time operating system (RTOS)
7. Lim, Y. W. (2014). Introduction to Embedded Systems.
8. DeepSea Developments. (2022). 15 real life embedded systems examples.
9. Burns, A., & Wellings, A. (2016). Real-Time Systems and Programming Languages. Addison-Wesley.
10. Google. (2018). Android Things Documentation. Google Developers.
11. Labrosse, J. J. (2009). MicroC/OS-II: The Real-Time Kernel. CMP Books.
12. Levis, P., Madden, S., & Gay, D. (2004). TinyOS: An operating system for sensor networks. Ambient Intelligence.
13. QNX Software Systems. (2018). QNX Neutrino RTOS Documentation.
14. Stallings, W. (2017). Operating Systems: Internals and Design Principles. Pearson.
15. Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems. Pearson.
16. Wind River Systems. (2019). VxWorks RTOS Product Overview.
17. Yaghmour, K. (2003). Building Embedded Linux Systems. O'Reilly Media.
18. Hildebrand, D. (1992). An architectural overview of QNX. Proceedings of the USENIX Workshop on Micro-Kernels and Other Kernel Architectures, 113–126.
19. Joseph, M., & Pandya, P. (1986). Finding response times in a real-time system. The Computer Journal, 29(5), 390–395.
20. Liu, C. L., & Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard-real-time environment. Journal of the ACM, 20(1), 46–61.
21. Sha, L., Rajkumar, R., & Lehoczky, J. P. (1990). Priority inheritance protocols: An approach to real-time synchronization. IEEE Transactions on Computers, 39(9), 1175–1185.
22. Wehmeyer, L., & Marwedel, P. (2007). Influence of memory hierarchies on predictability for time constrained embedded software. Journal of Systems Architecture, 53(11), 762–771.
23. Yoon, J., Kim, G., & Lee, K. (2013). Performance evaluation of FreeRTOS, QNX, and VxWorks for real-time embedded systems. International Journal of Advanced Computer Science and Applications, 4(4), 50–57.
24. David Kleidermacher, Mike Kleidermacher (2012). Embedded Systems Security: Practical Methods for Safe and Secure Software and Systems Development
25. Marwedel, P. (2010). Embedded system design: Embedded systems foundations of cyber-physical systems. Springer Science & Business Media.
26. Langner R. (2011) “Stuxnet: Dissecting a cyberwarfare weapon,” IEEE Security & Privacy, 9(3), 49–51.
27. Serpanos, D. N., & Voyiatzis, A. G. (2013). Security challenges in embedded systems. ACM Transactions on Embedded Computing Systems (TECS), 12(1s), 66.

28. Dorottya Papp, Zhendong Ma, Levente Buttyan (2015). Embedded Systems Security: Threats, Vulnerabilities, and Attack Taxonomy. Budapest University of Technology and Economics, Hungary.
29. Fiveable(2025). Embedded Systems Design, 18.1: Security threats and vulnerabilities in embedded systems
30. Splashtop (2025). Risks & Vulnerabilities of Unpatched Software
31. European Commission. (2022). Cyber Resilience Act: Security rules for digital products. Brussels.
32. Kandukuri, A. (2021). Secure Over-the-Air (OTA) Firmware Updates for IoT Devices. IEEE Internet of Things Journal.
33. Klein, G., Andronick, J., Elphinstone, K., et al. (2009). "seL4: Formal verification of an OS kernel." Proceedings of the 22nd ACM Symposium on Operating Systems Principles.
34. NIST. (2018). Platform Firmware Resiliency Guidelines (SP 800-193). National Institute of Standards and Technology.
35. Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero Trust Architecture (NIST SP 800-207). National Institute of Standards and Technology.
36. Sabt, M., Achemlal, M., & Bouabdallah, A. (2015). "Trusted Execution Environments: A
37. Taxonomy and Survey." ACM Computing Surveys.
38. Seacord, R. C. (2013). Secure Coding in C and C++. Addison-Wesley Professional.
39. Shostack, A. (2014). Threat Modeling: Designing for Security. Wiley.
40. Souppaya, M., & Scarfone, K. (2016). Guide to Enterprise Patch Management Technologies (NIST SP 800-40r3).
41. Xiao, L., Wan, X., Lu, X., & Zhang, Y. (2018). "IoT Security Techniques Based on Machine Learning." IEEE Signal Processing Magazine.